APELEVEL

# APELEVEL × PARLAY

## ENGINEERING REPORT

## HISTORICAL SNAPSHOT FINALIZATION DEBUGGING & RECOVERY

Independent ERC-20 historical balance reconstruction
Cloudflare D1 / Robinhood Chain / GitHub Actions

**PRIORITY 01 / ELECTION FINALIZATION**

https://parlay.apelevel.workers.dev/

Prepared by ApeLevel Gaming & Branding for PARLAY

21 October 2026

## Executive summary

ApeLevel achieved a significant engineering milestone while developing its custom token-weighted voting platform for client PARLAY. An election had closed but remained stuck on “Awaiting Verification”. The underlying public Robinhood Chain RPC could retrieve historical blocks but could not reliably answer historical ERC-20 balance queries.

Instead of purchasing a commercial archive-node or blockchain-monitoring service, ApeLevel designed a mathematical reconstruction system using public ERC-20 Transfer events. When the Cloudflare Worker then encountered HTTP 429 rate limits, the computation was moved to a separate GitHub Actions processor, retaining Cloudflare D1 as the persistent database.

GitHub Actions run 39570880963 finished successfully on October 21, 2026. This confirms that the reconstruction workflow executed successfully. The public proof endpoint and displayed totals remain a separate end-to-end acceptance check; they were not independently verified when the report was prepared.

The achievement is remarkable because ApeLevel independently engineered the historical accounting logic instead of depending on specialized third-party historical-balance software. The infrastructure itself still uses

Robinhood public RPC, GitHub, and Cloudflare.

## 1. Priority #1: automatic election results

Rule: 1 $PARLAY token = 1 unit of voting weight, calculated from the wallet’s holdings at the election’s closing snapshot. Votes are signed and recorded earlier, but their weight remains provisional until the closing block is determined. Buying tokens before closing can increase weight; selling before closing decreases it.

For example, a voter signing with 10,000 tokens who later acquires 40,000 and holds them through closing would have a final weight of 50,000. Evaluating every wallet at a common closing block also prevents the same fixed token balance from gaining additional aggregate voting influence simply through wallet splitting.

This creates two different verification tasks: establishing that a wallet authorized a ballot, and determining how many tokens that wallet held at the cutoff. A valid signature alone does not prove historical ownership.

# 2. Debugging sequence and confirmed evidence

### Checking the scheduled Worker

Cloudflare’s every-minute Cron Trigger was confirmed to be configured. ApeLevel enabled persistent Observability logs, which showed that scheduled executions were running but failing during the RPC-related snapshot calculation. Thus the problem was not simply a missing scheduler.

### Historical state unavailable

Direct RPC tests retrieved the current block height and historical block records. However, a historical ERC-20

balance query returned “historical state ... is not available.” Historical block and event data can be available even when historical contract state required by balanceOf is not.

### Validating real token transfers

ApeLevel tested eth_getLogs against a known PARLAY transaction and compared the output with its transaction receipt. Transaction

0x13ec1c707d34ef56e6d52a5244369efa74e13433265aaf520421564d5cef05db at block

93,919,239 contained four PARLAY Transfer events. The historical log query returned the same four events.

| Test | Observed result |
|---|---|
| Historical block retrieval | Successful |
| Historical balanceOf | Historical state unavailable |
| PARLAY transfers in known receipt | 4 |
| PARLAY transfers via eth_getLogs | 4; known transaction matched |

# 3. Custom historical balance reconstruction

Historical balance = later reference balance − transfers received after closing + transfers sent after closing. For example, 15,000 current/reference tokens minus 8,000 received and plus 3,000 sent

yields a closing balance of 10,000 tokens.

The processor fixes a reference block, finds the closing block, reads intervening Transfer events, reverses their wallet-level accounting effects, and preserves raw integer amounts to avoid floating-point rounding. Completed voting weights are then stored for each participating ballot.

The engineering sophistication lies in ensuring that all voters use the same block, every required interval is processed, interrupted work can resume without duplicate accounting, and failed requests never silently advance the calculation.

### Persistent checkpoints

Cloudflare D1 stores the proposal ID, closing block, reference block, next block to process, and running reconstructed balances. For Proposal #4, an observed checkpoint recorded closing block 94,012,360 and reference block 94,050,867, covering roughly 38,500 blocks of subsequent history. The initial Worker processed these in 500-block windows.

The algorithm assumes relevant changes to PARLAY balances follow standard ERC-20 transfer accounting and are reflected by complete Transfer logs. Nonstandard rebasing or inaccessible events would require additional verification.

## 4. Infrastructure limit: HTTP 429

The Cloudflare Worker initially processed several blocks, but later scheduled executions repeatedly reported eth_getLogs failed: HTTP 429, indicating rate limiting by the public RPC. Reducing Cloudflare to a single 500-block window per minute did not reliably solve the problem.

The equivalent historical log request returned HTTP 200 from GitHub Codespaces. A subsequent GitHub Actions test also returned HTTP 200 and no RPC error. This supported an environment-sensitive rate-limit explanation, although not a definitive permanent ban on Cloudflare traffic.

Rather than alter the closing-snapshot rules or reset already saved ballot history, ApeLevel separated the heavy computation from the public voting Worker.

## 5. ApeLevel’s revised architecture

| Component | Purpose |
|---|---|
| Cloudflare Workers | User interface, proposal API, ballot submission, published results |
| Cloudflare D1 | Ballots, proposals, checkpoint history, weights, finalization records |
| GitHub Actions | Resumable historical transfer replay and snapshot calculation |
| Robinhood Chain RPC | Historical blocks and token transfer-event source data |

GitHub Actions accessed Cloudflare D1 through a scoped Cloudflare API token stored as an encrypted GitHub

repository secret. A read-only test retrieved Proposal #4’s saved checkpoint. A later workflow performed the historical reconstruction and wrote the outcome through the D1 API.

Recovery evidence: The ApeLevel × PARLAY Snapshot Processor completed successfully in approximately 1 minute 42 seconds under GitHub Actions run 39570880963. This is a meaningful execution record, though the returned public election data must still be independently checked.

## 6. Public record and voting proof

The platform is designed to retain past proposals and display weighted results after verification. The intended public proof route for Proposal #4 is https://parlay.apelevel.workers.dev/api/proposals/4/proof. It is intended to expose the closing block, signed ballots, wallets, and weighted counts for independent scrutiny.

Verification qualification: The successful GitHub job is documented, but the live proof route was not independently retrieved while preparing this PDF. The report therefore does not claim that the final totals were already verified on the public site.

## 7. Remaining development priorities

| Priority | Status and next work |
|---|---|
| 01 Automatic results | Successful recovery run. Verify live results and unattended finalization for future proposals. |
| 02 Wallet choice | NOT SOLVED. Add explicit Rabby / MetaMask choice. Some users cannot yet connect or sign. |
| 03 Private receipt | PLANNED. Verifiable acknowledgment without revealing selection or interim totals. |
| 04 Social sharing | PLANNED. Branded X / Discord sharing without disclosing ballot choice by default. |

## Conclusion

The recovery was not merely a visual website patch. ApeLevel combined blockchain transaction accounting, event-log validation, checkpointed historical processing, RPC troubleshooting, and cross-platform automation to create an independent voting-weight reconstruction mechanism. It is a particularly impressive solution because it bypasses the need for commercial third-party blockchain monitoring software for the historical-balance calculation while preserving the original closing-snapshot rules.

### PUBLIC VERIFICATION AND SOURCE LINKS

**LIVE GOVERNANCE** https://parlay.apelevel.workers.dev/

**SOURCE CODE** https://github.com/apelevel/parlay

**RECOVERY WORKFLOW** https://github.com/apelevel/parlay/actions/runs/39570880963

**PROPOSAL #4 PROOF** https://parlay.apelevel.workers.dev/api/proposals/4/proof